Звіт

Лабораторна робота №4

Студента ПМІ-44

Стегнея Віктора

Варіант 15

Задана вибірка №3:

	q0	X11	X12	X21	X22	X31	X32	X33	Y1	Y2	Y3	Y4
1	1	5.05	8.65	7.75	6.975	4.879	3.501	5.967	254.621	98.145	119.406	117.683
2	2	5.052	8.7	7.78	6.955	4.886	3.553	5.978	198.163	73.368	92.651	90.123
3	3	5.055	8.745	7.80	6.95	4.897	3.611	5.984	187.411	91.084	87.691	83.576
4	4	5.06	8.75	7.82	6.945	4.916	3.652	5.987	167.197	123.567	78.793	74.789
5	5	5.063	9.8	7.845	6.925	4.938	3.723	5.996	166.547	163.813	79.497	74.316
6	6	5.064	10.25	7.851	6.895	4.947	3.758	5.999	153.789	261.378	77.082	72.817
7	7	5.067	11.85	7.852	6.865	4.954	3.784	5.976	110.926	355.579	67.758	77.425
8	8	5.07	12.87	7.853	6.854	4.967	3.809	5.964	151.381	440.432	51.956	89.519
9	9	5.075	14.90	8.854	6.856	4.978	3.825	5.958	187.364	336.283	91.123	121.374
10	10	5.08	16.91	8.855	6.855	4.984	3.845	5.937	236.123	223.657	112.859	149.173
11	11	5.085	18.92	9.856	6.856	4.987	3.851	5.916	292.341	118.624	153.717	184.136
12	12	5.09	15.92	10.86	6.865	4.996	3.8534	5.874	344.324	91.324	117.965	179.152
13	13	5.095	12.93	11.85	7.859	4.999	3.8536	5.842	426.939	68.926	155.912	201.239
14	14	5.1	11.93	12.87	7.876	4.976	3.854	5.814	477.128	44.675	169.359	225.482
15	15	5.125	9.935	11.89	7.895	4.964	3.856	5.756	505.327	29.367	192.924	240.976
16	16	5.135	8.941	9.925	7.925	4.958	3.859	5.718	558.386	18.567	218.549	275.846
17	17	5.15	7.945	8.945	7.945	4.937	3.867	5.671	618.859	23.932	247.354	316.124
18	18	5.153	6.951	7.945	7.951	4.916	3.879	5.629	895.737	35.124	284.167	363.928
19	19	5.157	5.965	6.95	6.955	4.874	3.886	5.567	906.168	61.946	316.375	403.153
20	20	5.2	4.965	5.965	6.975	4.842	3.897	5.486	885.761	121.387	341.326	431.195
21	21	5.25	3.974	4.975	7.001	4.814	3.916	5.452	790.639	310.519	375.651	471.588
22	22	5.3	2.981	5.000	7.125	4.756	3.938	5.501	723.784	485.142	446.856	436.847
23	23	5.315	3.985	6.975	7.145	4.718	3.947	5.554	731.438	588.125	548.314	441.842
24	24	5.325	4.990	7.955	7.165	4.671	3.954	5.621	721.321	683.435	644.716	439.425
25	25	5.35	5.995	8.945	7.195	4.629	3.967	5.658	691.845	772.834	729.942	422.147

26	26	5.353	7.997	9.935	7.209	4.567	3.978	5.712	508.614	880.562	849.316	435.954
27	27	5.357	9.001	10.92	7.225	4.482	3.984	5.753	429.956	687.987	748.231	450.492
28	28	5.4	10.94	11.89	7.25	4.452	3.987	5.781	330.129	488.951	647.987	454.897
29	29	5.425	12.90	12.86	7.975	4.364	3.996	5.802	127.152	385.494	442.967	458.289
30	30	5.445	10.88	14.85	7.955	4.326	3.999	5.825	78.654	211.209	232.856	172.164
31	31	5.465	8.944	15.85	7.95	4.264	3.976	5.845	52.145	196.197	115.632	153.356
32	32	5.475	6.780	12.85	7.945	4.184	3.964	5.851	86.243	87.325	93.135	127.168
33	33	5.485	6.764	10.85	7.925	4.156	3.958	5.854	126.345	64.615	77.824	106.123
34	34	5.495	6.568	8.865	7.895	4.136	3.937	5.856	132.879	52.534	63.453	82.659
35	35	5.497	6.437	6.859	7.865	4.129	3.916	5.854	167.156	32.178	52.167	93.834
36	36	5.5	5.325	4.876	7.854	4.116	3.874	5.856	170.531	66.176	42.836	91.345
37	37	5.515	5.206	2.895	7.853	4.098	3.842	5.859	184.243	70.364	37.192	96.841
38	38	5.525	5.149	1.925	7.855	4.0816	3.814	5.867	191.956	76.428	25.834	93.952
39	39	5.545	5.089	3.945	7.856	4.0686	3.756	5.879	216.829	83.475	50.985	109.463
40	40	5.575	4.933	4.953	7.865	4.0486	3.718	5.886	383.329	104.924	98.591	133.415
41	41	5.6	4.889	5.955	7.859	4.0246	3.671	5.005	279.421	184.183	102.861	108.613
42	42	5.65	3.935	6.975	7.876	4.0126	3.629	5.027	225.356	286.324	105.817	107.319
43	43	5.7	3.941	7.001	7.895	4.0114	3.567	5.049	176.578	366.457	78.473	82.263
44	44	5.745	2.945	7.125	7.925	4.0026	3.484	5.095	170.948	265.814	81.417	84.132
45	45	5.75	3.95	7.145	7.945	4.0019	3.452	5.189	158.334	184.549	78.653	81.953

Модель: мультиплікативна, реалізована через log1p / expm1

Задання вхідних даних

Вибірка задана csv файлом, вказаним вище, інші параметри прикріпляю:

```python
N1, N2, N3, M = 2, 2, 3, 4
P1, P2, P3 = 3, 3, 3
SIZE = 45
FAMILIES = ("laguerre", "hermite", "shifted_legendre")

data = load_and_normalize("data.csv", n1=N1, n2=N2, n3=N3, m=M, sample_size=SIZE)
results = fit_model(data["x1"], data["x2"], data["x3"], data["y"], p1=P1, p2=P2, p3=P3, families=FAMILIES)

print_results(data, results, FAMILIES)
plot_results(data["y"], results["y_pred"], data["y_names"])
```

✓ 0.3s

Результати

1. Вектор b у log-п росторі

	b
q1	0.163649
q2	0.124021
q3	0.123750
q4	0.109210
q5	0.108643
q6	0.154727
q7	0.196844
q8	0.228292
q9	0.198725
q10	0.161837
q11	0.191104
q12	0.202524
q13	0.229849
q14	0.241394
q15	0.247082
q16	0.265505
q17	0.293547
q18	0.407763
q19	0.421625
q20	0.435946
q21	0.469617
q22	0.490348
q23	0.494702
q24	0.490387
q25	0.502128
q26	0.542729
q27	0.496386
q28	0.425314
q29	0.269164
q30	0.143720
q31	0.112415
q32	0.094673
q33	0.083102
q34	0.080222
q35	0.087434
q36	0.094651
q37	0.098777
q38	0.096915
q39	0.122170
q40	0.223588
q41	0.177667
q42	0.182306
q43	0.206785
q44	0.160998
q45	0.119868

2. Lambda для X1:

	k0	k1	k2	k3
X11	0.429967	27.467375	-50.835908	22.920758
X12	0.429967	2.447259	-3.743777	1.118959

Lambda для X2

	k0	k1	k2	k3
X21	0.429967	-0.072549	-0.090227	-0.107855
X22	0.429967	-1.044153	0.560717	-0.470645

Lambda для X3

	k0	k1	k2	k3
X31	0.429967	0.594676	-0.368538	-0.531136
X32	0.429967	0.256412	-0.077090	0.017183
X33	0.429967	0.001750	-0.372536	-0.137357

3. Матриця A1:

	X11	X12
Y1	-1.202485	0.907847
Y2	-0.291124	1.119257
Y3	-1.020844	0.386458
Y4	-0.849049	0.556927

Матриця A2

	X21	X22
Y1	2.440301	-0.220556
Y2	1.458347	-0.230339
Y3	0.020197	-0.998467
Y4	-0.045461	-1.044073

Матриця A3

	X31	X32	X33
Y1	0.582511	1.008022	0.992522
Y2	0.736450	0.575548	-0.148524
Y3	0.722694	0.604648	0.756619
Y4	0.430854	0.650462	0.754306

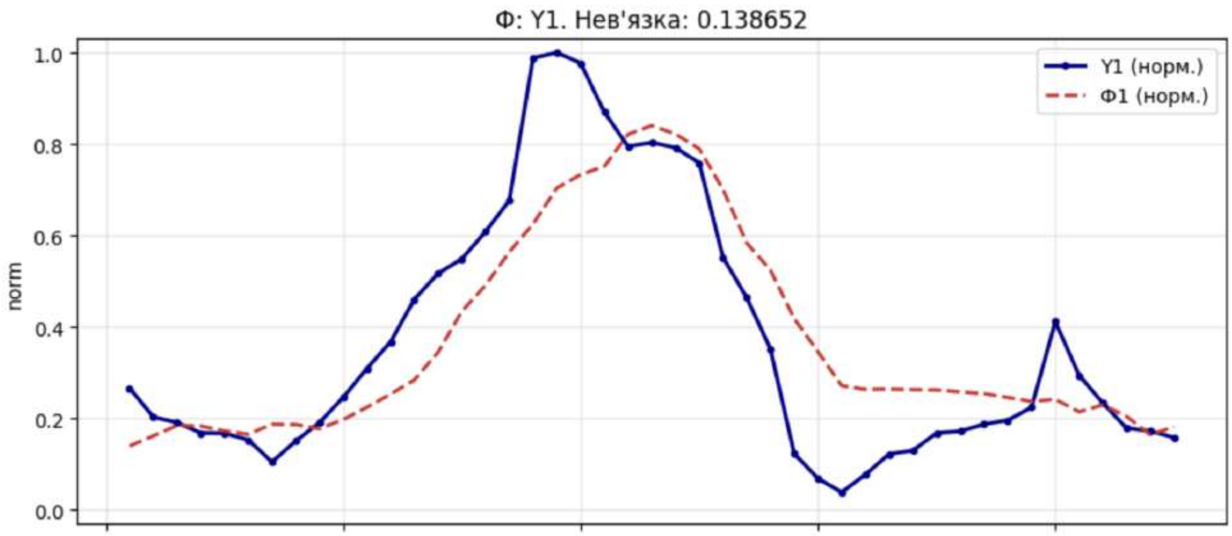
4. Частинні функції у логарифмічному просторі:

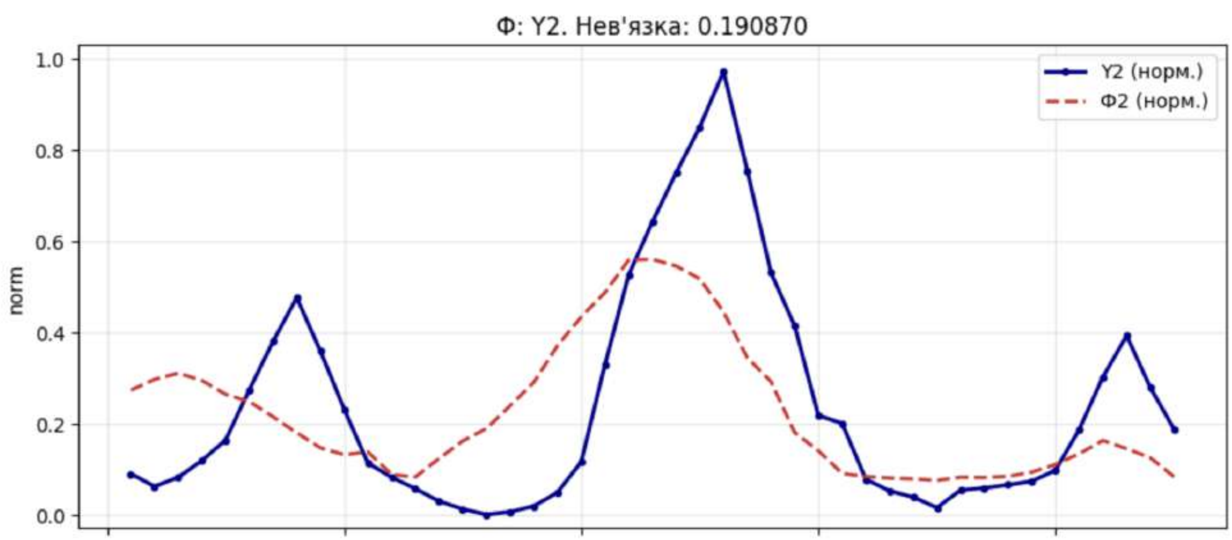
```
Для Y1:
ln(1 + Φ11) = −0.51702964 ln(1 + L0(X11)) + −33.02911910 ln(1 + L1(X11)) + 61.12944011 ln(1 + L2(X11))
    −27.56187830 ln(1 + L3(X11)) + 0.39034485 ln(1 + L0(X12)) + 2.22173787 ln(1 + L1(X12))
    −3.39877823 ln(1 + L2(X12)) + 1.01584365 ln(1 + L3(X12))
ln(1 + Φ12) = 1.04925028 ln(1 + H0(X21)) + −0.17704263 ln(1 + H1(X21)) + −0.22018069 ln(1 + H2(X21))
    −0.26319930 ln(1 + H3(X21)) + −0.09483187 ln(1 + H0(X22)) + 0.23029405 ln(1 + H1(X22))
    −0.12366948 ln(1 + H2(X22)) + 0.10380345 ln(1 + H3(X22))
ln(1 + Φ13) = 0.25046089 ln(1 + L*0(X31)) + 0.34640531 ln(1 + L*1(X31)) + −0.21467759 ln(1 + L*2(X31))
    −0.30939281 ln(1 + L*3(X31)) + 0.43341648 ln(1 + L*0(X32)) + 0.25846926 ln(1 + L*1(X32))
    −0.07770874 ln(1 + L*2(X32)) + 0.01732072 ln(1 + L*3(X32)) + 0.42675215 ln(1 + L*0(X33))
    0.00173689 ln(1 + L*1(X33)) + −0.36975044 ln(1 + L*2(X33)) + −0.13632944 ln(1 + L*3(X33))

Для Y2:
ln(1 + Φ21) = −0.12517402 ln(1 + L0(X11)) + −7.99642279 ln(1 + L1(X11)) + 14.79957266 ln(1 + L2(X11))
    −6.67279170 ln(1 + L3(X11)) + 0.48124419 ln(1 + L0(X12)) + 2.73911243 ln(1 + L1(X12))
    −4.19024936 ln(1 + L2(X12)) + 1.25240245 ln(1 + L3(X12))
ln(1 + Φ22) = 0.62704174 ln(1 + H0(X21)) + −0.10580232 ln(1 + H1(X21)) + −0.13158203 ln(1 + H2(X21))
    −0.15729035 ln(1 + H3(X21)) + −0.09903837 ln(1 + H0(X22)) + 0.24050932 ln(1 + H1(X22))
    −0.12915515 ln(1 + H2(X22)) + 0.10840791 ln(1 + H3(X22))
ln(1 + Φ23) = 0.31664960 ln(1 + L*0(X31)) + 0.43794903 ln(1 + L*1(X31)) + −0.27140993 ln(1 + L*2(X31))
    −0.39115532 ln(1 + L*3(X31)) + 0.24746713 ln(1 + L*0(X32)) + 0.14757779 ln(1 + L*1(X32))
    −0.04436924 ln(1 + L*2(X32)) + 0.00988958 ln(1 + L*3(X32)) + −0.06386061 ln(1 + L*0(X33))
    −0.00025991 ln(1 + L*1(X33)) + 0.05533069 ln(1 + L*2(X33)) + 0.02040079 ln(1 + L*3(X33))

Для Y3:
ln(1 + Φ31) = −0.43892952 ln(1 + L0(X11)) + −28.03989250 ln(1 + L1(X11)) + 51.89550845 ln(1 + L2(X11))
    −23.39850792 ln(1 + L3(X11)) + 0.16616423 ln(1 + L0(X12)) + 0.94576209 ln(1 + L1(X12))
    −1.44681136 ln(1 + L2(X12)) + 0.43243013 ln(1 + L3(X12))
ln(1 + Φ32) = 0.00868391 ln(1 + H0(X21)) + −0.00146526 ln(1 + H1(X21)) + −0.00182228 ln(1 + H2(X21))
    −0.00217832 ln(1 + H3(X21)) + −0.42930830 ln(1 + H0(X22)) + 1.04255195 ln(1 + H1(X22))
    −0.55985752 ln(1 + H2(X22)) + 0.46992308 ln(1 + H3(X22))
ln(1 + Φ33) = 0.31073481 ln(1 + L*0(X31)) + 0.42976846 ln(1 + L*1(X31)) + −0.26634019 ln(1 + L*2(X31))
    −0.38384882 ln(1 + L*3(X31)) + 0.25997890 ln(1 + L*0(X32)) + 0.15503923 ln(1 + L*1(X32))
    −0.04661252 ln(1 + L*2(X32)) + 0.01038959 ln(1 + L*3(X32)) + 0.32532135 ln(1 + L*0(X33))
    0.00132407 ln(1 + L*1(X33)) + −0.28186786 ln(1 + L*2(X33)) + −0.10392655 ln(1 + L*3(X33))

Для Y4:
ln(1 + Φ41) = −0.36506329 ln(1 + L0(X11)) + −23.32113655 ln(1 + L1(X11)) + 43.16215688 ln(1 + L2(X11))
    −19.46083774 ln(1 + L3(X11)) + 0.23946069 ln(1 + L0(X12)) + 1.36294585 ln(1 + L1(X12))
    −2.08501225 ln(1 + L2(X12)) + 0.62317877 ln(1 + L3(X12))
ln(1 + Φ42) = −0.01954677 ln(1 + H0(X21)) + 0.00329818 ln(1 + H1(X21)) + 0.00410181 ln(1 + H2(X21))
    0.00490321 ln(1 + H3(X21)) + −0.44891725 ln(1 + H0(X22)) + 1.09017121 ln(1 + H1(X22))
    −0.58542939 ln(1 + H2(X22)) + 0.49138714 ln(1 + H3(X22))
ln(1 + Φ43) = 0.18525326 ln(1 + L*0(X31)) + 0.25621850 ln(1 + L*1(X31)) + −0.15878616 ln(1 + L*2(X31))
    −0.22884222 ln(1 + L*3(X31)) + 0.27967747 ln(1 + L*0(X32)) + 0.16678653 ln(1 + L*1(X32))
    −0.05014434 ln(1 + L*2(X32)) + 0.01117681 ln(1 + L*3(X32)) + 0.32432695 ln(1 + L*0(X33))
    0.00132002 ln(1 + L*1(X33)) + −0.28100628 ln(1 + L*2(X33)) + −0.10360888 ln(1 + L*3(X33))
```

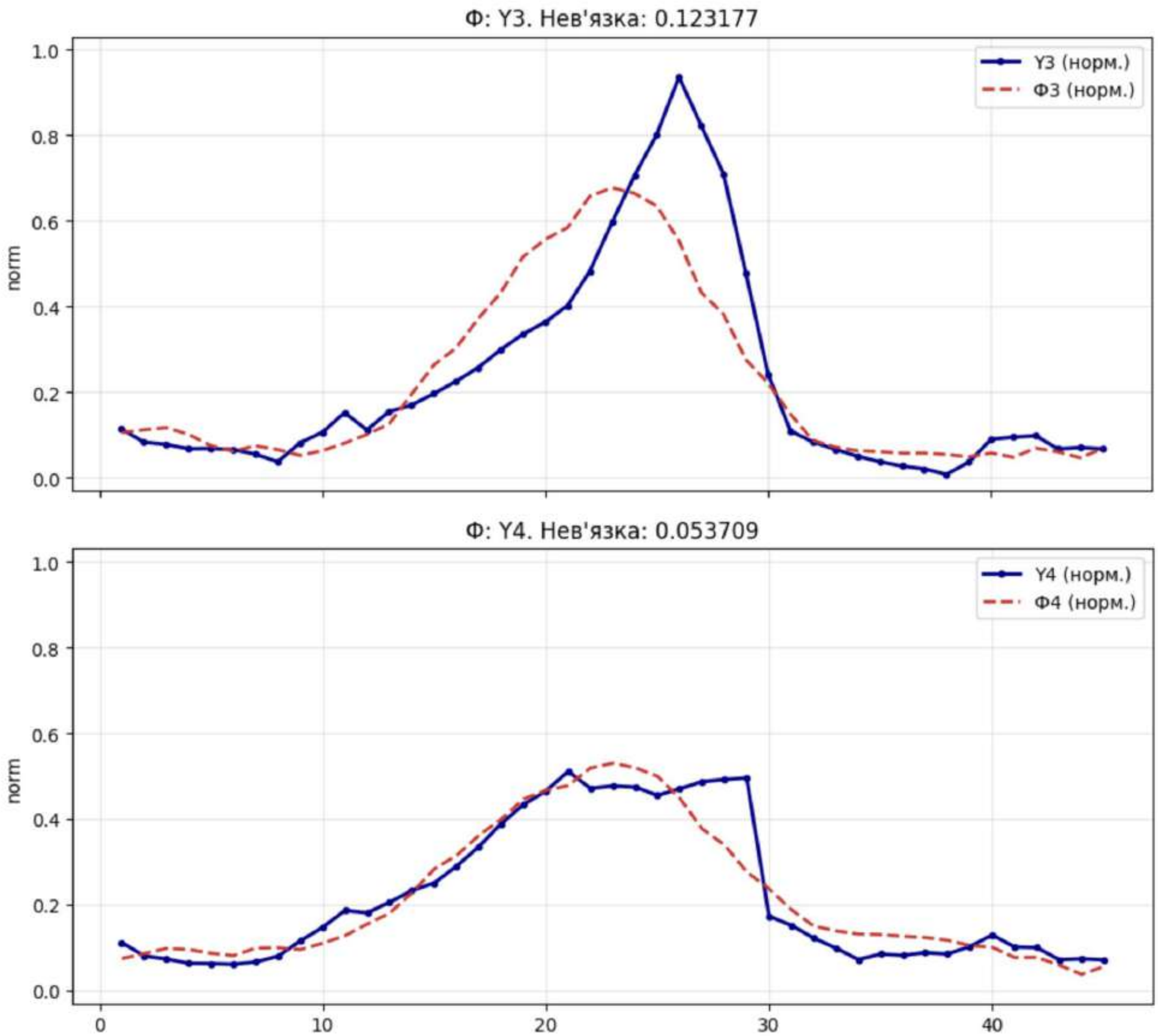
5. Підсумкові функції:

```
ln(1 + Φ1) = −0.24075088 ln(1 + Φ11) + 0.23049220 ln(1 + Φ12) + 1.00528115 ln(1 + Φ13)
ln(1 + Φ2) = −0.65256706 ln(1 + Φ21) + 0.37410426 ln(1 + Φ22) + 1.19255416 ln(1 + Φ23)
ln(1 + Φ3) = −0.27653573 ln(1 + Φ31) + 0.10752832 ln(1 + Φ32) + 1.12974638 ln(1 + Φ33)
ln(1 + Φ4) = −0.08757926 ln(1 + Φ41) + 0.06616765 ln(1 + Φ42) + 1.01794677 ln(1 + Φ43)

Матриця C
             c1        c2        c3
 Y1 | -0.240751  0.230492  1.005281
 Y2 | -0.652567  0.374104  1.192554
 Y3 | -0.276536  0.107528  1.129746
 Y4 | -0.087579  0.066168  1.017947
```

6. Порівняння реальних і відновлених значень Y:







Код реалізації

```
def laguerre(x, degree):
    if degree == 0:
        return np.ones_like(x)
    if degree == 1:
        return 1.0 - x

    l_prev2 = np.ones_like(x)
    l_prev1 = 1.0 - x
    for n in range(2, degree + 1):
        current = ((2 * n - 1 - x) * l_prev1 - (n - 1) * l_prev2) / n
        l_prev2, l_prev1 = l_prev1, current
    return l_prev1



def hermite(x, degree):
```

```python
    if degree == 0:
        return np.ones_like(x)
    if degree == 1:
        return 2.0 * x

    h_prev2 = np.ones_like(x)
    h_prev1 = 2.0 * x
    for n in range(2, degree + 1):
        current = 2.0 * x * h_prev1 - 2.0 * (n - 1) * h_prev2
        h_prev2, h_prev1 = h_prev1, current
    return h_prev1


def shifted_legendre(x, degree):
    z = 2.0 * x - 1.0
    if degree == 0:
        return np.ones_like(z)
    if degree == 1:
        return z

    p_prev2 = np.ones_like(z)
    p_prev1 = z
    for n in range(2, degree + 1):
        current = ((2 * n - 1) * z * p_prev1 - (n - 1) * p_prev2) / n
        p_prev2, p_prev1 = p_prev1, current
    return p_prev1


def basis_value(x, degree, family):
    if degree == 0:
        return np.full_like(x, 0.5, dtype=float)

    if family == "laguerre":
        raw = laguerre(x, degree)
    elif family == "hermite":
        raw = hermite(x, degree)
    elif family == "shifted_legendre":
        raw = shifted_legendre(x, degree)
    else:
        raise ValueError(f"Невідомий базис: {family}")

    min_value = float(raw.min())
    max_value = float(raw.max())
    diff = max_value - min_value
    if diff == 0:
        return np.full_like(x, 0.5, dtype=float)
    return (raw - min_value) / diff
```

```python
def basis_symbol(family):
    return {
        "laguerre": "L",
        "hermite": "H",
        "shifted_legendre": "L*",
    }[family]


def safe_log1p(values):
    clipped = np.maximum(values, -0.999999999)
    return np.log1p(clipped)


def solve_lstsq(a, b):
    return np.linalg.lstsq(a, b, rcond=None)[0]


def calc_b_values(y):
    average_level = (y.max(axis=1) + y.min(axis=1)) / 2.0
    return safe_log1p(average_level).reshape(-1, 1)


def build_log_design_matrix(x_groups, degrees, families):
    columns = []
    for x_matrix, degree, family in zip(x_groups, degrees, families):
        for j in range(x_matrix.shape[1]):
            for k in range(degree + 1):
                columns.append(safe_log1p(basis_value(x_matrix[:, j], k, family)))
    return np.column_stack(columns)


def split_lambda_coefficients(lambdas, dims, degrees):
    matrices = []
    start = 0
    for dim, degree in zip(dims, degrees):
        width = dim * (degree + 1)
        matrices.append(lambdas[start:start + width].reshape(dim, degree + 1))
        start += width
    return matrices


def calculate_log_psi(lambda_matrix, x, degree, family):
    log_psi = np.zeros((x.shape[0], x.shape[1]))
    for j in range(x.shape[1]):
        for k in range(degree + 1):
            log_psi[:, j] += lambda_matrix[j, k] * safe_log1p(basis_value(x[:, j], k, family))
    return log_psi
```

```python
def incoherence(y_pred, y_true):
    return np.sqrt(np.mean((y_pred - y_true) ** 2, axis=0))


def denormalize(matrix, y_min, y_max):
    return matrix * (y_max - y_min) + y_min


def build_comparison_table(y_names, y_real, y_pred):
    header = ["q"]
    for name in y_names:
        header.extend([f"{name}_real", f"{name}_pred"])

    rows = []
    for idx in range(y_real.shape[0]):
        row = [str(idx + 1)]
        for column in range(y_real.shape[1]):
            row.extend([
                f"{y_real[idx, column]:.6f}",
                f"{y_pred[idx, column]:.6f}",
            ])
        rows.append(row)
    return header, rows


def fit_model(x1, x2, x3, y, p1, p2, p3, families):
    tz_matrix = build_log_design_matrix((x1, x2, x3), (p1, p2, p3), families)
    b_vector = calc_b_values(y)
    lambda_all = solve_lstsq(tz_matrix, b_vector)
    lambda_x1, lambda_x2, lambda_x3 = split_lambda_coefficients(
        lambda_all,
        (x1.shape[1], x2.shape[1], x3.shape[1]),
        (p1, p2, p3),
    )

    psi_1_log = calculate_log_psi(lambda_x1, x1, p1, families[0])
    psi_2_log = calculate_log_psi(lambda_x2, x2, p2, families[1])
    psi_3_log = calculate_log_psi(lambda_x3, x3, p3, families[2])

    psi_1 = np.expm1(psi_1_log)
    psi_2 = np.expm1(psi_2_log)
    psi_3 = np.expm1(psi_3_log)

    z_y = safe_log1p(y)
    a1 = solve_lstsq(safe_log1p(psi_1), z_y)
    a2 = solve_lstsq(safe_log1p(psi_2), z_y)
    a3 = solve_lstsq(safe_log1p(psi_3), z_y)
```

```
phi_1_log = safe_log1p(psi_1) @ a1
phi_2_log = safe_log1p(psi_2) @ a2
phi_3_log = safe_log1p(psi_3) @ a3

phi_1 = np.expm1(phi_1_log)
phi_2 = np.expm1(phi_2_log)
phi_3 = np.expm1(phi_3_log)

c_matrix = np.zeros((3, y.shape[1]))
z_pred = np.zeros_like(y)
y_pred = np.zeros_like(y)
for idx in range(y.shape[1]):
    current = np.column_stack((phi_1_log[:, idx], phi_2_log[:, idx], phi_3_log[:, idx]))
    c_matrix[:, idx] = solve_lstsq(current, z_y[:, idx]).ravel()
    z_pred[:, idx] = current @ c_matrix[:, idx]
    y_pred[:, idx] = np.expm1(z_pred[:, idx])

return {
    "tz_matrix": tz_matrix,
    "b_vector": b_vector,
    "lambda_x1": lambda_x1,
    "lambda_x2": lambda_x2,
    "lambda_x3": lambda_x3,
    "psi_1": psi_1,
    "psi_2": psi_2,
    "psi_3": psi_3,
    "a1": a1,
    "a2": a2,
    "a3": a3,
    "phi_1": phi_1,
    "phi_2": phi_2,
    "phi_3": phi_3,
    "phi_1_log": phi_1_log,
    "phi_2_log": phi_2_log,
    "phi_3_log": phi_3_log,
    "c_matrix": c_matrix,
    "z_pred": z_pred,
    "y_pred": y_pred,
    "rmse": incoherence(y_pred, y),
}
```